Module 7 Week A — Core Skills Drill: Fine-Tuning Prep

This drill is due the evening of the concept session (Sunday). It moves you from "I read about `Trainer` and `datasets`" to "I can prepare a labeled dataset for fine-tuning and configure a `Trainer` so that training would run." You will not run training in the drill — that is tomorrow's lab. The drill targets the mechanical configuration steps the lab assumes.

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. **Accept the assignment**
2. **Clone your repo:**

   ```
   git clone <your-repo-url>
   cd <your-repo-name>
   ```

3. **Install dependencies:**

   ```
   pip install -r requirements.txt
   ```

   This installs `transformers`, `datasets`, `evaluate`, `accelerate`, `torch` (CPU build), and `scikit-learn`. The first three are already installed if you completed Module 6; `evaluate` and `accelerate` are new in Module 7.

4. **Create a working branch:**

   ```
   git checkout -b drill-7a-finetune-prep
   ```

All your work goes in `drill.py`. The starter file contains function signatures and docstrings — implement each function as described below.

The drill uses a small fixture CSV (`fixtures/tiny_app_reviews.csv`, 21 labeled app reviews with three sentiment classes) so the drill runs in seconds. The lab uses the full curated AARSynth app-review dataset (~7,500 examples).

Task 1: Build a Dataset from a CSV

Implement `make_dataset(csv_path, test_size, seed)`:

- Load the CSV at `csv_path` (columns: `text`, `label`) into a Pandas DataFrame.
- Convert the DataFrame into a Hugging Face `Dataset` (use `Dataset.from_pandas`).
- Split into `train` and `test` using `train_test_split` with the passed `test_size` and `seed`.
- Return the resulting `DatasetDict`.

Why this matters: Every fine-tuning workflow starts with a `DatasetDict` of splits. The lab's pipeline assumes you can produce one from a CSV.

```
def make_dataset(csv_path: str, test_size: float, seed: int) -> DatasetDict:
    """
    Load a CSV with text and label columns; split into train/test.

    Returns a DatasetDict with keys "train" and "test".
    """
    # TODO: read CSV into a DataFrame
    # TODO: convert to a Hugging Face Dataset
    # TODO: split with the passed test_size and seed
    # TODO: return the DatasetDict
```

Task 2: Tokenize the Dataset

Implement `tokenize_dataset(ds_dict, tokenizer_name, max_length)`:

- Load an `AutoTokenizer` from the passed `tokenizer_name` (e.g., `"distilbert-base-uncased"`).
- Tokenize both splits using `dataset.map` with `batched=True`.
- Apply `truncation=True` and `max_length=max_length` in the tokenizer call.
- Do **not** apply padding here — padding will be applied dynamically per batch in the lab.
- Return the tokenized `DatasetDict`. Both splits should have `input_ids` and `attention_mask` columns.

```
def tokenize_dataset(ds_dict: DatasetDict, tokenizer_name: str, max_length: int)
    """
    Tokenize all splits in a DatasetDict using the named tokenizer.

    The result has input_ids and attention_mask columns added to each split.
    """
    # TODO: load the tokenizer
    # TODO: define a tokenize function that calls the tokenizer with truncation a
    # TODO: apply ds_dict.map with batched=True
    # TODO: return the tokenized dataset dict
```

Common pitfall: if you forget `batched=True`, your tokenize function receives one example at a time, which is much slower (and the resulting dataset structure may be wrong). Make sure `batched=True` is on.

Task 3: Build TrainingArguments

Implement `make_training_args(output_dir, lr, epochs, batch_size, seed)`:

- Return a `TrainingArguments` object with:
  - `output_dir=output_dir`
  - `learning_rate=lr`
  - `num_train_epochs=epochs`
  - `per_device_train_batch_size=batch_size`
  - `per_device_eval_batch_size=batch_size`
  - `seed=seed`
  - `eval_strategy="epoch"`
  - `save_strategy="epoch"`
  - `logging_steps=50`

```
def make_training_args(output_dir: str, lr: float, epochs: int, batch_size: int,
    """
    Build a TrainingArguments with the standard fine-tuning configuration.
    """
    # TODO: return TrainingArguments(...) with the kwargs above
```

Why this matters: the lab will pass these `TrainingArguments` directly into `Trainer`. Hard-coded values in `make_training_args` would mean the lab cannot vary hyperparameters cleanly — and the autograder catches this.

Task 4: Build compute_metrics

Implement `compute_metrics(eval_pred)`:

- Unpack `eval_pred` into `(logits, labels)`.
- Convert `logits` (shape `(N, num_classes)`) into class predictions with `np.argmax(logits, axis=1)`.
- Compute accuracy and **macro**-F1 (use `f1_score(..., average="macro")` from sklearn).
- Return `{"accuracy": ..., "macro_f1": ...}`.

```
def compute_metrics(eval_pred):
    """
    Convert model logits + labels into a metrics dict.

    Returns: {"accuracy": float, "macro_f1": float}
    """
    # TODO: unpack eval_pred into logits and labels
    # TODO: argmax logits over axis 1 to get class predictions
    # TODO: compute accuracy
    # TODO: compute macro-F1 (average="macro")
    # TODO: return both as a dict
```

Common pitfall: `f1_score` with no `average` argument returns a per-class array. With `average="weighted"` it weights by class frequency (wrong for class-imbalanced data). **Always use `average="macro"` for this drill and the lab.**

Submission

1. Make sure `pytest tests/ -v` runs and reports passing or failing tests cleanly (every test should not fail with `ImportError` or `SyntaxError`).
2. Commit your work and push the branch.
3. Open a PR from your `drill-7a-finetune-prep` branch into `main`.
4. The autograder will run automatically. All 8 tests should pass.

Your PR description must include:

1. One sentence per task explaining what you did (4 sentences total).
2. Paste your PR URL into TalentLMS → Module 7 → Core Skills Drill 7A to submit this assignment.

Troubleshooting

- **`ImportError: cannot import name 'TrainingArguments'`** — make sure you've installed the latest `transformers` and `accelerate`. Run `pip install -r requirements.txt` again.
- **`Dataset.from_pandas` complains about an `__index_level_0__` column** — pass `preserve_index=False` to `Dataset.from_pandas(df, preserve_index=False)`.
- **Tests look for `input_ids` and `attention_mask` and don't find them** — you probably forgot `batched=True` in `dataset.map`.
- **Macro-F1 test fails on perfect predictions but accuracy passes** — you used `average="weighted"` instead of `average="macro"`.